ECE 420 Final Project

Anna Bagdishyan

***Abstract*—The AXI GPIO IP block was implemented and verified in Vivado using VHDL. The block provides control of outputs through an AXI4-Lite interface. The design utilizes a system controller for clocking and reset, and a testbench verifies its functionality. Simulation results show the correct AXI read/write behavior and expected operation.**

Keywords—AXI GPIO, AXI4-Lite, VHDL, IP Block

I. Introduction

This project involved implementing the AXI GPIO IP block in a VHDL design. The AXI GPIO was used to create a design that could read and write data through an AXI4-Lite interface while also controlling the direction of GPIO pins. Because AXI GPIO operates through the AXI protocol, part of this project involved learning how AXI works in order to design and verify the implementation.

II. IP Block Overview and Use Cases

The AXI GPIO IP block provides a general-purpose input/output interface through an AXI4-Lite interface. It supports reading and writing GPIO data, controlling whether pins act as inputs or outputs, and the option to enable interrupts. Because of this, AXI GPIO is useful when a design needs a way to read or drive signals within an AXI-based design. Common uses include driving LEDs and reading switches or buttons.
One advantage of AXI GPIO is that it is simple to configure and easy to integrate into a design. It also offers several IP configuration options. However, AXI GPIO is best suited for basic status and control tasks rather than more complex designs. Applications that require more intricate timing and communication protocols may require more specialized IP blocks, such as AXI Timer or AXI UART.

III. Interface and IP Options

The IP options for AXI GPIO includes all inputs, which makes the channel input only, and all outputs, which makes it output only. GPIO Width selects the number of GPIO bits from 1 to 32. The default output value sets the initial output value, the default tri state value sets the initial direction. Direction is controlled by the GPIO_TRI register, where a 0 configures it as output and a 1 as input [1]. Enable dual channel adds a second GPIO channel, and the GPIO2 settings become available. Enable interrupt adds interrupt support. For this project, the IP was configured as a single channel, 4-bit GPIO with interrupts disabled. Neither all inputs nor all outputs was selected, so the direction could be decided through GPIO_TRI to allow for testing both input and output operation.

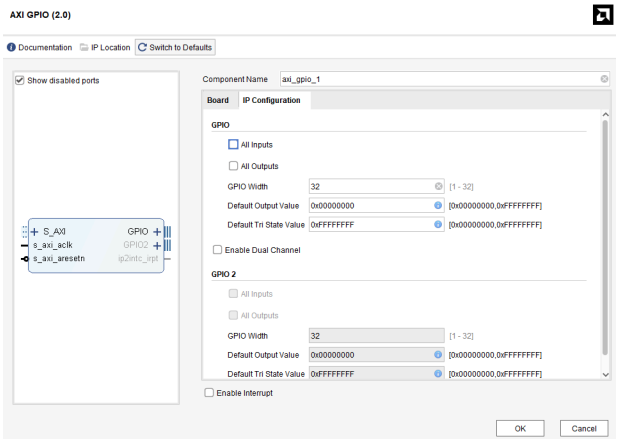

Figure 1 - IP Block Options for AXI GPIO

AXI GPIO operates from an externally supplied AXI clock and does not generate its own clock or reset. In the design, the AXI GPIO clock is clk1, which is generated by the clock wizard. The AXI GPIO reset is s_axi_aresetn, which is an active low reset. The clocking and reset for the AXI GPIO are handled externally by the clock wizard.

IV. Design

This design used the AXI GPIO IP block. The top entity includes the input clock and reset, the AXI4-Lite interface signals, GPIO connections, and the outputs locked and clk_out. The design instantiates a system controller and an AXI GPIO block. The system controller, which was provided, generates the internal clock used by the AXI GPIO and also produces the internal reset signal. The AXI reset signal is active low and is generated by inverting the system controller reset output. The AXI GPIO block is connected to the AXI interface and operates using the generated clock. Because the design is not restricted to only input or only output, the direction of the pins can be changed through the GPIO_TRI register. A 0 configures a pin as an output while a 1 configures it as an input.

A testbench was created to verify the functionality of the design. The testbench generated a clock, applied a reset, and then performed AXI write and read operations. The tests included configuring the GPIO as outputs, writing different output values, switching the GPIO to input mode, and reading back an applied input value. The testbench specifically verified that the GPIO could be configured in output mode, that values 1010 and 0101 were correctly written to the outputs, that the GPIO could be switched to input mode, and that an applied input value of 1100 could be read back correctly. If any test failed, an error message was reported and the simulation ended. After fixing test 5, all tests passed, and the "TEST PASSED" was displayed in the Tcl console.

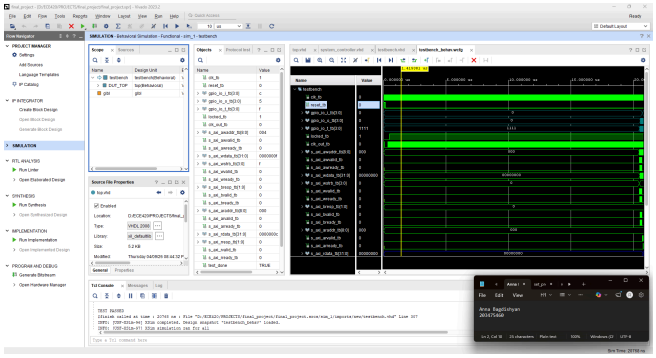

Figure 2 - This screenshot shows a behavioral simulation for the design. All testbench and DUT signals were added to the waveform. This screenshot also shows the "TEST PASSED" message in the Tcl Console.

One challenge was determining how to verify the AXI GPIO, because the AXI interface requires multiple signals across multiple clock cycles. This made verification less straightforward, but the issue was addressed by developing a better understanding of AXI so that writing values to the GPIO outputs and reading values from the GPIO inputs could be tested correctly.

Another challenge was related to timing during the AXI read test. Test 5 initially failed because the read operation occurred too soon after applying the input. This was fixed by inserting additional clock cycles before the read to allow the value more time to reach the expected result. After this change, all tests passed.

V. Inquiry and Application of New Knowledge

For this project, I was not initially familiar with the AXI4-Lite interface or the AXI GPIO IP block. Although I already understood the concept of GPIO, the AXI-based implementation was more complex because of the structured read and write communication protocol and multiple signals involved.

To develop an understanding of these topics, I referred to the AXI documentation and the AMD product guide. The documentation helped me understand the basic read and write processes, while the product guide explained the purpose of the GPIO data and tri-state registers. Specifically, it clarified how the GPIO_TRI register determines whether a pin is an input or output.

I then applied this knowledge during both the design and testing stages of the project. Understanding the AXI4-Lite protocol allowed me to correctly implement and test the read and write operations in the testbench, and understanding the GPIO registers helped me verify that the design worked as intended. The combination of information from the documentation with the design helped me achieve passing results for all test cases.

VI. Reflection

This project helped me develop a better understanding of AXI-based protocols in VHDL. I learned that AXI4-Lite uses multiple signals to manage read and write operations and gained a better understanding of how AXI GPIO is used in hardware design. This project also reinforced concepts from previous assignments, especially the importance of testing both timing and functionality. This project helped me better understand the design and verification of AXI-based designs.

VII. Conclusion

For this project, an AXI GPIO IP block was successfully designed and verified using a testbench. The design showed correct functionality for both writing output values and reading input values through the AXI interface. Although several challenges were encountered, they were resolved and all test cases passed. The project provided experience in learning a new interface and integrating an IP block into a VHDL design.

References

[1] *"AXI GPIO LogiCORE IP Product Guide (PG144)." AMD Technical Information Portal, 30 Jan. 2026,* docs.amd.com/r/en-US/pg144-axi-gpio/AXI-GPIO-3-State-Control-Register-GPIOx_TR

[2] *ARM Ltd. "AMBA AXI Protocol Specification."*

[3] *Tracton, Philip. Week Ten Lecture Slides. ECE 420, Digital Systems Design with Programmable Logic*